Data Mining

MLOPS

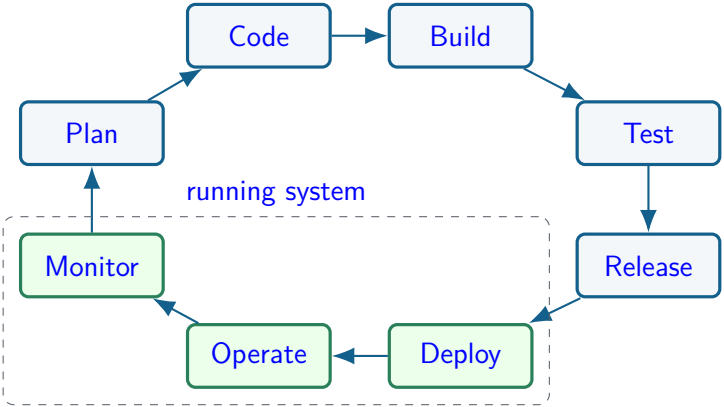
Claudio Sartori

DISI
Department of Computer Science and Engineering – University of Bologna, Italy
claudio.sartori@unibo.it

What DevOps means

- DevOps is a way of building and running software as one continuous system, not as isolated jobs.
- It joins three concerns:
  - development: writing and changing applications,
  - operations: deploying, securing, and maintaining them,
  - feedback: measuring what happens in production.
- Main objective: deliver changes faster, with less risk, and recover quickly when something fails.

The DevOps lifecycle



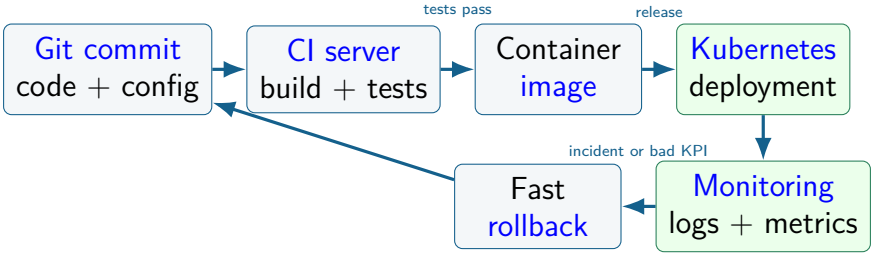
- The cycle matters because deployment, operations, and monitoring continuously inform the next change.

Core practices behind DevOps

- Continuous Integration (CI): every change is built and tested automatically.
- Continuous Delivery / Deployment (CD): releases follow a repeatable pipeline.
- Infrastructure as Code (IaC): servers, networks, and policies are described as versioned files.
- Observability: logs, metrics, traces, and alerts show system health.
- Automation: reduce manual steps that are slow or error-prone.
- Shared ownership: the same team cares about both feature delivery and production reliability.

Without DevOps	separate silos, handovers, slow releases, weak feedback
With DevOps	integrated pipeline, fast validation, controlled releases, continuous learning

A practical pipeline example

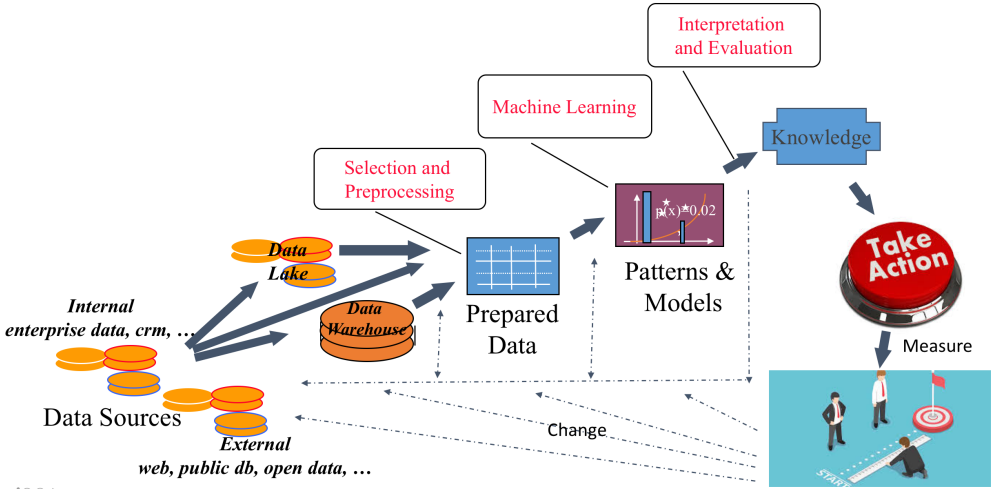


- A small change starts in version control, is validated in CI, packaged as an image, and deployed automatically.
- If the new version causes errors or performance drops, the team uses monitoring data to roll back or fix forward.

Takeaway

DevOps is not mainly about tools. It is about using automation, measurement, and collaboration to make software systems reliable and easy to change.

The Data Mining Process



Data Pipeline (recall from Machine Learning workflows)

Definition

A data pipeline is an automated sequence of processes that moves data from its source through a set of transformations to produce data that is ready for analysis, training, or serving in a machine learning system.

Data Pipeline - II

Typical stages

- Ingestion
  - collecting data from sources (databases, sensors, APIs, logs)
- Processing
  - cleaning, filtering, transforming, and aggregating data
- Storage
  - storing intermediate or processed data (data lakes, warehouses)
- Feature preparation
  - producing features used by ML models
- Delivery
  - providing data to training pipelines, analytics, or production services

Data Pipeline - III

Key property

A data pipeline is typically automated, repeatable, and scalable, enabling reliable data preparation for machine learning and data-driven systems.

What is MLOps?

- Machine Learning Operations (MLOps)
- Discipline for managing the lifecycle of machine learning systems
- Extends DevOps principles to data-driven systems
- Goals
  - Improve collaboration between teams
    - data scientists
    - ML engineers
    - software engineers
    - operations teams
  - Automate and standardize the ML lifecycle
  - Enable reliable and scalable ML deployment

Why MLOps is Needed

- ML systems are different from traditional software
  - behavior depends on data
  - models degrade over time
  - reproducibility is difficult
- Challenges
  - data drift
  - concept drift
  - reproducibility
  - monitoring in production
- MLOps addresses these challenges with
  - automation
  - monitoring
  - governance

Modern MLOps Pipeline

A production ML system typically includes:

- Data management
- Feature engineering
- Model training
- Experiment tracking
- Model evaluation
- Model registry
- Deployment
- Monitoring
- Continuous retraining

Data Versioning

- Data is a first-class citizen in MLOps
- Version datasets to ensure reproducibility
  - track dataset evolution
  - reproduce training experiments
- Typical tools
  - DVC
  - LakeFS
  - Delta Lake

Data Validation

- Validate datasets before training
- Detect issues such as
  - missing values
  - schema changes
  - distribution shifts
- Prevent corrupted data entering pipelines
- Tools
  - Great Expectations
  - TensorFlow Data Validation
  - Evidently

Feature Stores

- Centralized storage of ML features
- Benefits
  - reuse features across models
  - avoid duplication of feature pipelines
  - ensure consistency between training and inference
- Feature stores manage
  - feature definitions
  - feature lineage
  - feature serving
- Examples
  - Feast
  - Tecton
  - Hopsworks

Experiment Tracking

- Track experiments during model development
- Record
  - hyperparameters
  - datasets
  - model versions
  - evaluation metrics
- Enables reproducibility and comparison
- Tools
  - MLflow
  - Weights & Biases
  - Neptune

Hyperparameter Optimization

- Selecting optimal hyperparameters improves performance
- Common methods
  - Grid search
  - Random search
  - Bayesian optimization
  - Hyperband / ASHA
- Tools
  - Optuna
  - Ray Tune
  - Hyperopt

Training Pipelines

- Training pipelines automate model creation
- Steps typically include
  - data preprocessing
  - feature engineering
  - model training
  - evaluation
- Pipelines improve
  - reproducibility
  - scalability

Model Registry

Central repository for trained models

- Stores
  - model artifacts
  - model metadata
  - model versions
  - evaluation metrics
- Enables lifecycle management
  - staging
  - production
  - archived models
- Examples
  - MLflow Model Registry
  - SageMaker Model Registry
  - Vertex AI Registry

Model Packaging

- Package models for deployment
- Common approach
  - serialize model
  - containerize with Docker
- Benefits
  - portability
  - reproducibility

Model Serving

- Deploy models as services
- Types of inference
  - batch inference
  - online inference
  - streaming inference
- Popular serving frameworks
  - KServe
  - Seldon
  - BentoML
  - TorchServe

Infrastructure and Orchestration

- Large ML systems require orchestration
- Pipeline orchestration tools
  - Kubeflow Pipelines
  - Apache Airflow
  - Prefect
  - Dagster
- Container orchestration
  - Kubernetes

CI/CD/CT for Machine Learning

- Continuous Integration (CI)
  - automatically test code and pipelines
- Continuous Deployment (CD)
  - deploy models automatically
- Continuous Training (CT)
  - retrain models when new data arrives
  - retrain when performance degrades

Monitoring ML Systems

- Monitoring is essential in production
- Monitor
  - model performance
  - prediction latency
  - system availability
- Detect issues early

Data and Concept Drift

- Data drift
  - input data distribution changes
- Concept drift
  - relationship between inputs and outputs changes
- Drift detection enables
  - retraining
  - alerting

Responsible AI in MLOps

- Production ML systems must be trustworthy
- Key aspects
  - fairness
  - explainability
  - transparency
  - auditability
- Tools
  - SHAP
  - LIME
  - Fairness toolkits

Explainability in MLOps

- Machine learning models are often complex and opaque
- Explainability helps understand
  - why a model produced a prediction
  - how features influence decisions
  - whether the model behaves as expected
- Explainability supports
  - debugging models
  - validating model assumptions
  - increasing trust in deployed systems

SHAP: Model Explanations

- SHAP (SHapley Additive exPlanations)
- Based on Shapley values from cooperative game theory
- Provides explanations by
  - assigning a contribution value to each feature
  - measuring the impact of features on predictions
- Typical uses in MLOps
  - global feature importance analysis
  - explanation of individual predictions
  - monitoring changes in feature importance

LIME: Local Model Explanations

- LIME (Local Interpretable Model-agnostic Explanations)
- Explains predictions locally
- Key idea
  - approximate a complex model
  - using a simple interpretable model
  - around a specific prediction
- Typical uses in MLOps
  - investigating unexpected predictions
  - debugging model behaviour
  - explaining individual decisions

Fairness in Machine Learning

- Machine learning models may produce biased decisions
- Bias may arise from
  - unbalanced training data
  - historical biases in data
  - inappropriate feature selection
- Fairness analysis evaluates
  - differences in model outcomes across groups
  - discrimination risks
  - ethical implications of automated decisions

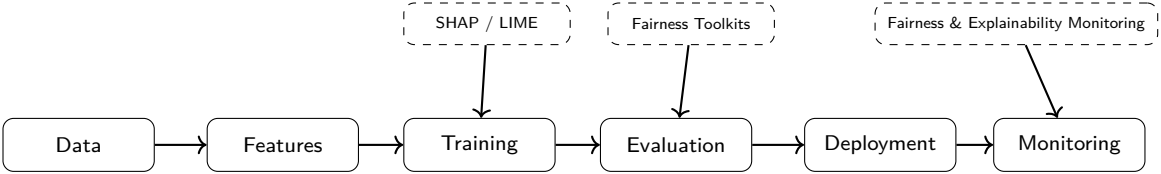
Fairness Toolkits

- Toolkits help detect and mitigate bias in models
- Provide
  - fairness metrics
  - bias detection tools
  - bias mitigation algorithms
- Examples
  - IBM AI Fairness 360
  - Fairlearn
  - Google What-If Tool
- Typical use in MLOps
  - model validation before deployment
  - auditing models for fairness
  - monitoring bias in production

Explainability and Fairness in the MLOps Pipeline

- Model development
  - understand model behaviour during training
  - detect unexpected feature influence
  - tools: SHAP, LIME
- Model validation
  - verify interpretability requirements
  - check fairness metrics across demographic groups
  - tools: Fairlearn, AI Fairness 360
- Model deployment
  - document model explanations
  - support regulatory and auditing requirements
- Model monitoring
  - detect changes in feature importance
  - monitor fairness metrics over time

Explainability and Fairness in the MLOps Pipeline



- Explainability tools help interpret model behaviour during training and evaluation
- Fairness toolkits detect bias before deployment
- Monitoring ensures explanations and fairness remain stable in production

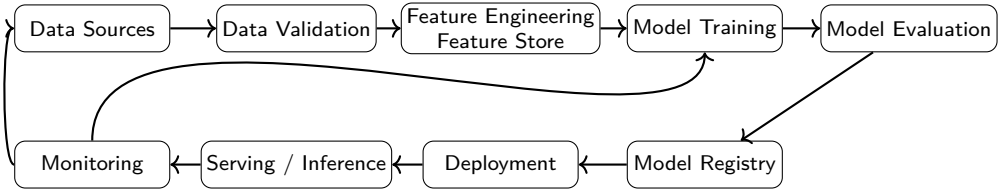
LLMOps

- LLMOps extends MLOps to generative AI systems
- Key challenges
  - prompt management
  - evaluation of generative outputs
  - hallucination detection
  - safety guardrails
- Often integrated with
  - Retrieval Augmented Generation (RAG)

Example Modern MLOps Stack

- Version Control
  - Git
- Data Versioning
  - DVC
- Experiment Tracking
  - MLflow
  - Weights & Biases
- Orchestration
  - Kubeflow
  - Airflow
- Serving
  - KServe
  - BentoML
- Monitoring
  - Evidently
  - Prometheus + Grafana

Modern MLOps Architecture Diagram



How to Read the Modern MLOps Architecture

- Data layer
  - data is collected, validated, and transformed into features
  - feature stores help maintain training-serving consistency
- Model development layer
  - models are trained and evaluated
  - experiments and model versions should be tracked
- Deployment layer
  - validated models are registered and promoted to deployment
  - inference may be batch, online, or streaming
- Operations layer
  - monitoring tracks model quality, drift, latency, and failures
  - feedback from monitoring can trigger retraining
- Key idea
  - modern MLOps is a closed-loop system with continuous improvement

Differences between CRISP-DM and MLOps I

- Purpose and Focus
  - CRISP-DM Structured methodology for data mining projects, covering the entire project lifecycle
  - MLOps Focuses on the operationalization and management of machine learning models in production

Differences between CRISP-DM and MLOps II

- Activities
  - CRISP-DM Involves business understanding, data preparation, modeling, evaluation, and deployment
  - MLOps Involves model deployment, continuous integration, continuous delivery, monitoring, and managing the machine learning model lifecycle
- Methodological vs. Operational
  - CRISP-DM Methodological framework for structuring the data mining process
  - MLOps Operational practices and tools for managing machine learning models in production

Differences between CRISP-DM and MLOps III

- Timing
  - CRISP-DM Applied throughout the entire data mining project
  - MLOps Becomes prominent once the machine learning model is ready for deployment and continues throughout its operational lifecycle
- Collaboration
  - CRISP-DM Emphasizes collaboration between business stakeholders, data scientists, and domain experts
  - MLOps Encourages collaboration between data scientists, developers, and operations teams for deployment and maintenance

CRISP-DM and MLOps – Summary

- Distinct concepts addressing different aspects of the data mining and machine learning lifecycle
- CRISP-DM guides the entire data mining project
- MLOps specifically addresses the operational aspects of managing machine learning models in production
- They are complementary in an end-to-end workflow

Bibliography

- Raj, E. (2021).
  Engineering MLOps : Rapidly Build, Test, and Manage Production-ready Machine Learning Life Cycles at Scale.
  Packt Publishers.